
Hugo Phone

Manual de implementación

y referencia técnica

*Recreación del juego **Hugo** (DOS, 1996) controlado por teléfono analógico vía tonos DTMF, replicando la experiencia televisiva de los años 90, en una arquitectura completamente offline basada en Raspberry Pi, Asterisk y un bridge nativo en Rust.*



Beto

Los Ángeles, Biobío, Chile

Versión 1.0 ■ 27 de mayo de 2026

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos del proyecto	1
1.3. Decisiones de diseño	2
1.3.1. ¿Por qué Raspberry Pi 4?	2
1.3.2. ¿Por qué Asterisk y no algo más ligero?	2
1.3.3. ¿Por qué Rust para el bridge?	2
1.3.4. ¿Por qué DOSBox clásico?	2
1.4. Estructura del manual	3
1.5. Convenciones tipográficas	3
2. Arquitectura del sistema	4
2.1. Visión general	4
2.2. Flujo de datos	4
2.3. Componentes lógicos	5
2.3.1. Asterisk	5
2.3.2. hugo-bridge (Rust)	5
2.3.3. DOSBox	5
2.3.4. Servicios del sistema	5
2.4. Topología de red	5
2.5. Decisiones de seguridad	6
3. Lista de materiales	7
3.1. Componentes esenciales	7
3.1.1. Detalles por componente	7
3.2. Componentes opcionales	9
3.3. Plan de crecimiento: multijugador	9
3.4. Comparativa con alternativas	9
4. Fase 1: Preparación de la Raspberry Pi	11
4.1. Objetivo	11
4.2. Materiales	11
4.3. Paso 1: Flashear la microSD	11
4.4. Paso 2: Primer arranque	12
4.5. Paso 3: Configuración de red	12
4.6. Paso 4: Conexión por SSH	12
4.7. Paso 5: Actualización del sistema	13
4.8. Paso 6: Clonar el repositorio del proyecto	13
4.9. Paso 7: Ejecutar el instalador	13
4.10. Paso 8: Configurar secretos	14
4.11. Paso 9: Verificación	14

4.12. Resolución de problemas	14
4.12.1. Asterisk no inicia	14
4.12.2. La Pi se reinicia sin causa aparente o presenta lag aleatorio	15
4.12.3. SSH no responde	15
4.12.4. El instalador falla durante la compilación del bridge	15
5. Fase 2: Prueba SIP con softphone	16
5.1. Objetivo	16
5.2. Paso 1: Instalar Linphone	16
5.3. Paso 2: Crear cuenta SIP en Linphone	16
5.4. Paso 3: Verificar registro desde Asterisk	16
5.5. Paso 4: Llamada de prueba al 9000	17
5.6. Paso 5: Probar DTMF	17
5.7. Resolución de problemas	18
5.7.1. Linphone no se registra	18
5.7.2. La llamada al 9000 se corta inmediatamente	18
5.7.3. Conecta pero no se escucha el beep	18
5.7.4. No aparecen UserEvent al marcar DTMF	18
5.8. Cierre de fase	18
6. Fase 3: DOSBox y Hugo	20
6.1. Objetivo	20
6.2. Estado legal del juego	20
6.3. Paso 1: Conseguir Hugo	20
6.3.1. Caso A: ZIP con HUGO.EXE suelto	20
6.3.2. Caso B: ISO de CD-ROM (Hugo 4)	21
6.4. Paso 2: Instalar DOSBox	21
6.5. Paso 3: Instalador del juego (sólo Caso B / ISO)	21
6.6. Paso 4: Verificar la configuración de DOSBox	21
6.7. Paso 5: Lanzar Hugo	22
6.8. Paso 6: Arranque automático del juego	22
6.8.1. Habilitar autologin en TTY1	22
6.8.2. Ejecutar DOSBox al iniciar sesión	22
6.9. Paso 7: Prueba de jugabilidad	23
6.10. Paso 8: Salir limpiamente	23
6.11. Resolución de problemas	23
6.11.1. <code>startx</code> no encuentra display o falla	23
6.11.2. Hugo se cuelga en la intro o las cinemáticas	24
6.11.3. El audio se entrecorta	24
6.11.4. El juego pide CD pero el ISO está montado	24
6.11.5. Hugo arranca pero no responde al teclado	24
6.12. Cierre de fase	24
7. Fase 4: El bridge en Rust	25
7.1. Objetivo	25
7.2. Estructura del código	25

7.3.	Dependencias y filosofía de diseño	25
7.4.	El protocolo AMI	26
7.5.	Anatomía del módulo <code>ami.rs</code>	26
7.5.1.	<code>run</code> : bucle principal con reconexión	26
7.5.2.	<code>connect_and_loop</code> : conexión, login y bucle de eventos	27
7.5.3.	<code>handle_event</code> : filtrado y acción	27
7.5.4.	<code>read_message</code> : parser del wire format	28
7.6.	El módulo <code>keyboard.rs</code>	28
7.6.1.	Creación del teclado virtual	29
7.6.2.	Inyección de pulsación	29
7.7.	El módulo <code>watcher.rs</code>	29
7.8.	Verificación práctica	30
7.9.	El flujo completo con <code>softphone</code>	30
7.10.	Calibración	31
7.11.	Activar como servicio	31
7.12.	Resolución de problemas	31
7.12.1.	<code>Permission denied</code> sobre <code>/dev/uinput</code>	31
7.12.2.	El bridge se conecta pero no recibe eventos	32
7.12.3.	Los DTMF llegan al bridge pero Hugo no responde	32
7.12.4.	Cargo no encuentra dependencias durante la compilación	32
7.13.	Cierre de fase	32
8.	Fase 5: Conectar el teléfono real	33
8.1.	Objetivo	33
8.2.	Conceptos previos	33
8.2.1.	Qué es un puerto FXS	33
8.2.2.	Auto-dial al descolgar	33
8.3.	Conexiones físicas	33
8.4.	Paso 1: Descubrir la IP del HT801	34
8.4.1.	Método A: IVR del propio HT801	34
8.4.2.	Método B: panel del router	34
8.5.	Paso 2: Acceder al panel web del HT801	34
8.6.	Paso 3: Configurar la cuenta SIP	34
8.7.	Paso 4: Configurar Auto-Dial	35
8.8.	Paso 5: Verificar registro	35
8.9.	Paso 6: La prueba final	36
8.10.	Resolución de problemas	36
8.10.1.	El teléfono no da tono al descolgar	36
8.10.2.	Hay tono pero al descolgar no marca el 9000	36
8.10.3.	Conecta pero no se escucha el beep	36
8.10.4.	Los DTMF no producen acción en Hugo	37
8.10.5.	La llamada se cae sola después de algunos minutos	37
9.	Fase 6: Modo multijugador (futuro)	38
9.1.	Visión	38

9.2. Cambios de hardware	38
9.3. Cambios en Asterisk	38
9.3.1. pjsip.conf: nuevas extensiones	38
9.3.2. confbridge.conf: salas de conferencia	38
9.3.3. extensions.conf: lógica de cola	39
9.4. Lógica de turnos	39
9.5. Audio del juego en la conferencia	39
9.5.1. Camino A: ALSA loopback	39
9.5.2. Camino B: PipeWire	40
9.6. Topología física	40
9.7. Funcionalidades opcionales	40
9.7.1. Identificación del jugador	40
9.7.2. Marcador / ranking	40
9.7.3. Display de marcador en pantalla	40
9.7.4. Streaming a Twitch o YouTube	41
9.8. Estimación de esfuerzo	41
9.9. Cierre	41
10. Referencia técnica	42
10.1. Mapeo DTMF → teclado	42
10.2. Archivos clave del sistema	42
10.3. Servicios y comandos útiles	42
10.3.1. Gestión de Asterisk	42
10.3.2. Gestión del bridge	43
10.3.3. Recompilar el bridge	43
10.3.4. Inspección del teclado virtual	43
10.3.5. Verificación del HT801	43
10.4. Estructura del archivo <code>config.toml</code>	43
10.5. Nombres de teclas soportados	44
10.6. Anatomía del protocolo AMI	44
10.6.1. Banner inicial	44
10.6.2. Login	44
10.6.3. Respuesta exitosa de login	44
10.6.4. UserEvent producido por el dialplan	45
10.7. Glosario	45
10.8. Referencias externas	45
10.9. Créditos	46

Introducción

1.1 Motivación

A inicios de los años noventa, varios canales de televisión emitieron programas en horario infantil donde los espectadores controlaban un personaje en pantalla *en directo* a través del teclado de su teléfono. El más famoso fue **Hugo**, un pequeño troll noruego creado por la compañía danesa ITE Media, que debía rescatar a su familia (Hugolina y sus hijos) de la malvada bruja Scylla. La transmisión llegó a más de cuarenta países y en Chile pasó por varios canales abiertos.

El mecanismo era simple en apariencia y sofisticado en infraestructura: la productora del programa ponía a disposición una línea telefónica de pago, los niños llamaban, esperaban su turno escuchando música y el audio del juego, y al recibirlo controlaban a Hugo durante alrededor de un minuto presionando dígitos del teclado:

- **2** para saltar o subir
- **4** para moverse a la izquierda
- **6** para moverse a la derecha
- **8** para agacharse o bajar
- **5** para acciones especiales

Cada pulsación generaba un tono *DTMF* (*Dual-Tone Multi-Frequency*) que la central del estudio decodificaba, traducía a un comando del juego y enviaba al PC que ejecutaba la versión interactiva de Hugo. La latencia —comparable a la actual de un partido de fútbol transmitido por streaming— formaba parte del encanto y del desafío.

Treinta años después, todos los componentes de aquel sistema están disponibles como software libre o hardware de bajo costo, lo que permite reconstruir la experiencia completa de forma doméstica, offline y a una fracción del costo original. Este manual documenta paso a paso cómo hacerlo.

1.2 Objetivos del proyecto

1. Ejecutar **Hugo** (versión DOS, 1996) en una Raspberry Pi 4 (2 GB) con salida HDMI a un televisor.
2. Conectar un **teléfono analógico DTMF** físico al sistema mediante un adaptador FXS (Grandstream HT801).
3. Implementar una **centralita telefónica local** en la misma Pi con Asterisk, sin requerir conexión a Internet ni servicios de telefonía externos.

4. Desarrollar un **punto (bridge)** en Rust que traduzca los tonos DTMF capturados por Asterisk a pulsaciones de teclado inyectadas al emulador DOSBox.
5. Documentar el sistema con un nivel de detalle que permita replicarlo, mantenerlo y extenderlo a un escenario *multijugador* estilo televisión.

1.3 Decisiones de diseño

1.3.1 ¿Por qué Raspberry Pi 4?

La Pi 4 es lo suficientemente potente para correr Asterisk, DOSBox y el bridge simultáneamente, sin dejar de ser un *appliance* compacto, silencioso, de bajo consumo (5–7 W) y con salida HDMI directa. El modelo de 2 GB es la opción más económica disponible y deja margen de RAM tras cargar todos los servicios.

1.3.2 ¿Por qué Asterisk y no algo más ligero?

Asterisk es la implementación de PBX más usada y mejor documentada del ecosistema libre. Trae decodificación nativa de DTMF (RFC 2833 e *inband*), *dialplan* extensible, *Manager Interface* (AMI) para integración con software externo y soporte de *ConfBridge* para conferencias *multiparty*. Alternativas como *baresip* o *kamailio* son más livianas, pero no incluyen la decodificación DTMF de alto nivel ni el *dialplan* que aquí se necesita.

1.3.3 ¿Por qué Rust para el bridge?

El *bridge* es el componente que recibe los eventos de DTMF desde Asterisk y los traduce a teclas virtuales mediante el subsistema *uinput* del kernel Linux. Las propiedades deseables son:

- **Latencia mínima** y predecible: el usuario percibe cualquier retraso entre pulsar el teléfono y ver moverse al personaje.
- **Bajo consumo de RAM**: la Pi de 2 GB no tiene mucho margen una vez cargado DOSBox.
- **Estabilidad**: el servicio debe correr indefinidamente sin reinicios.
- **Sin dependencias de runtime**: ideal un binario único, sin entornos virtuales ni intérpretes.

Rust cumple las cuatro propiedades sin esfuerzo. El binario final pesa pocos megabytes y consume del orden de 3–5 MB de RAM en operación.

1.3.4 ¿Por qué DOSBox clásico?

DOSBox (el proyecto original, paquete *dosbox* en Debian/Raspberry Pi OS) es maduro, está empaquetado en los repos oficiales de Trixie y soporta sin problemas todo lo que Hugo necesita: VGA, Sound Blaster 16, montaje de imágenes ISO con *imgmount*, y un núcleo *dynamic* con buen rendimiento en ARM. Los *forks* más modernos (DOSBox-Staging, DOSBox-X) aportan shaders OpenGL y KMSDRM más limpio, pero no son estrictamente necesarios para el motor

sencillo de Hugo y, al momento de escribir este manual, `dosbox-staging` no está disponible en los repositorios estándar de la Pi.

1.4 Estructura del manual

El documento se organiza en tres bloques:

Bloque conceptual (capítulos 1–3): introducción, arquitectura general del sistema y lista detallada de materiales (*Bill of Materials*).

Bloque de implementación (capítulos 4–8): seis fases secuenciales, cada una con objetivo claro, pasos verificables y *troubleshooting*. Cada fase se construye sobre la anterior.

Bloque de referencia (capítulos 9–10): visión futura del modo multijugador y referencia técnica completa (protocolos AMI, mapeo DTMF, comandos útiles).

Nota

Las fases están diseñadas para validar incrementalmente. Si una fase falla, no se debe avanzar a la siguiente: el sistema solo se vuelve más difícil de depurar al sumar componentes sobre una base que aún no funciona.

1.5 Convenciones tipográficas

- Los comandos de `shell` y nombres de archivos se muestran en monoespaciado.
- Los *términos técnicos* introducidos por primera vez aparecen en cursiva.
- Las cajas destacadas siguen este esquema:

Nota

Las cajas de tipo *Nota* aportan contexto adicional útil pero no crítico.

Advertencia

Las cajas de tipo *Advertencia* señalan situaciones donde un error puede romper el sistema o requerir reinstalación.

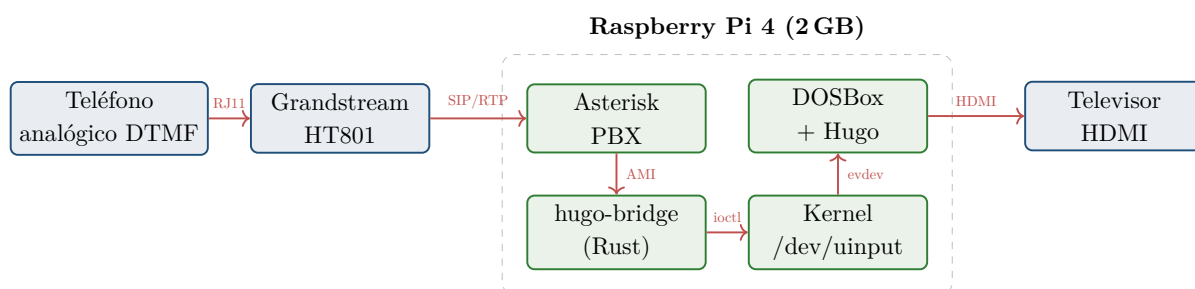
Tip

Las cajas de tipo *Tip* contienen atajos, trucos o decisiones aprendidas a partir de errores comunes.

Arquitectura del sistema

2.1 Visión general

El sistema completo se compone de tres dominios físicos y cinco dominios lógicos coexistiendo dentro de una sola Raspberry Pi:



2.2 Flujo de datos

Una pulsación del dígito **2** en el teléfono recorre el siguiente camino:

1. El teléfono genera el tono DTMF correspondiente al **2** (composición de 697 Hz + 1336 Hz) y lo envía como audio analógico al puerto FXS del HT801.
2. El HT801 detecta el tono y, según su configuración (*DTMF method: RFC 2833*), lo encapsula como evento RTP *out-of-band* y lo envía por SIP a Asterisk.
3. Asterisk recibe el evento RFC 2833 y lo expone al *dialplan* en ejecución como dígito leído por la función `Read()`.
4. El *dialplan* dispara un `UserEvent` con nombre `HugoKey` y campo `Key=2`, emitido por la *Asterisk Manager Interface* (AMI) sobre TCP en `127.0.0.1:5038`.
5. El proceso `hugo-bridge`, suscripto al AMI, parsea el evento, consulta el *keymap* configurado y obtiene la tecla destino (`KEY_UP`).
6. Mediante `ioctl(2)` sobre `/dev/uinput`, el bridge simula una pulsación física (`press + sync + sleep + release + sync`) sobre un teclado virtual previamente creado.
7. El kernel propaga el evento por `evdev` al subsistema de input, donde es leído por la biblioteca `SDL2` que `DOSBox` utiliza para entrada.
8. `DOSBox` traduce el código de tecla a una interrupción 16h de teclado IBM PC, que Hugo lee dentro de su *event loop*.
9. Hugo actualiza la posición del personaje y renderiza el siguiente frame por VGA emulado, que `DOSBox` vuelca a la pantalla HDMI.

La latencia total de extremo a extremo está dominada por la red SIP local y el ciclo de renderizado del juego. En pruebas internas, oscila entre 5 y 30 ms, perfectamente jugable.

2.3 Componentes lógicos

2.3.1 Asterisk

Centralita PBX de código abierto. Aquí cumple cuatro roles:

- **Registro SIP:** acepta el registro del HT801 como extensión 101, y opcionalmente softphones (Linphone) como extensiones de prueba.
- **Decodificación DTMF:** extrae los dígitos de RFC 2833 o *inband* y los entrega al dialplan.
- **Dialplan:** programa que define qué pasa con cada llamada. En Hugo Phone es un bucle infinito que lee un dígito y emite un `UserEvent`.
- **Manager Interface (AMI):** socket TCP en 127.0.0.1:5038 donde clientes externos se autentican y reciben eventos en tiempo real.

2.3.2 hugo-bridge (Rust)

Binario nativo escrito en Rust, ejecutándose como servicio `systemd`. Componentes internos:

- **Cliente AMI propio:** implementación TCP/texto del protocolo Asterisk Manager, sin dependencias de *crates* de terceros. Cerca de 150 líneas de código en `src/ami.rs`.
- **Teclado virtual:** *wrapper* sobre el crate `uinput`, crea un dispositivo de entrada que el kernel expone con nombre `hugo-phone-virtual-keyboard`.
- **Hot-reload de configuración:** *watcher* basado en `notify` que detecta cambios al archivo `config.toml` y actualiza el *keymap* y otros parámetros sin reiniciar el servicio.
- **Reconexión automática:** si la conexión con Asterisk cae, el bridge reintenta con *backoff* exponencial hasta 30 segundos.

2.3.3 DOSBox

Emulador DOS clásico (paquete `dosbox` de Debian) que provee CPU x86, VGA y Sound Blaster 16 virtualizados. Se lanza dentro de un servidor X mínimo (`xinit + openbox`) en modo *fullscreen*. El ISO del juego se monta como unidad CD-ROM mediante `imgmount` y Hugo arranca automáticamente desde el script `[autoexec]` del archivo de configuración.

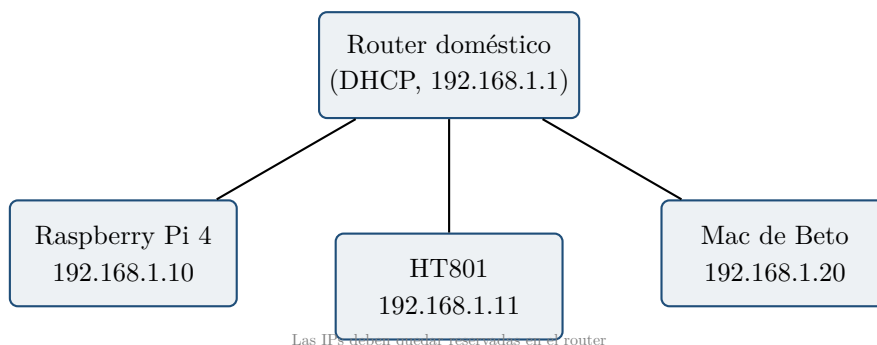
2.3.4 Servicios del sistema

2.4 Topología de red

El sistema completo vive en una LAN doméstica con tres equipos que se comunican:

Servicio	Inicio	Función
asterisk.service	systemd	PBX SIP
hugo-bridge.service	systemd (after asterisk)	DTMF → teclado
DOSBox	.bashrc en TTY1	Ejecuta Hugo
Autologin tty1	raspi-config	Arranque sin teclado

Tabla 2.1: Servicios persistentes en la Raspberry Pi.

**Tip**

Reservar IPs estáticas en el router (asociadas a la dirección MAC del dispositivo) es esencial. Si el HT801 cambia de IP entre reinicios, Asterisk no podrá enviarle paquetes y la línea queda muda.

2.5 Decisiones de seguridad

El sistema está diseñado para una red doméstica de confianza:

- Asterisk *no* expone su puerto 5060 al exterior. El router debe **no** hacer port-forward.
- AMI escucha solo en 127.0.0.1, accesible únicamente por procesos locales.
- Las contraseñas SIP y AMI son aleatorias y se cambian al instalar (los valores “cambiamе-*” deben ser reemplazados).
- El usuario del bridge corre como root porque necesita acceso a /dev/uinput. Alternativamente se puede usar el grupo input con una regla udev.

Advertencia

Si por algún motivo se decide exponer el sistema a Internet (por ejemplo, para permitir a amigos conectarse remotamente), se debe usar SIP sobre TLS, contraseñas robustas y, preferentemente, una VPN. SIP en texto plano hacia Internet es uno de los vectores de ataque más comunes para fraude telefónico.

Capítulo 3

Lista de materiales

3.1 Componentes esenciales

Los siguientes componentes son indispensables para el sistema en su configuración mínima (un solo teléfono). Precios referenciales en pesos chilenos a mediados de 2026.

Componente	Modelo sugerido	Precio CLP
Single Board Computer	Raspberry Pi 4 Modelo B (2 GB)	\$85.000
Fuente de poder	USB-C oficial Raspberry Pi 15 W (5 V/3 A)	\$15.000
Almacenamiento	MicroSD SanDisk 32 GB Clase 10 / A1	\$10.000
Disipación térmica	Caja con ventilador para Pi 4	\$12.000
Cable de video	Cable micro-HDMI a HDMI, 1.5 m	\$5.000
Adaptador telefónico	Grandstream HT801 (1 puerto FXS)	\$40.000
Cable telefónico	Cable RJ11, 2 m	\$2.000
Cable de red	Patch cord Ethernet Cat5e, 1 m	\$3.000
Teléfono analógico	Cualquier teléfono fijo con DTMF (años 90+)	—
Televisor/monitor	Equipo con entrada HDMI	—
Total esenciales		~\$172.000

Tabla 3.1: Bill of materials esencial.

3.1.1 Detalles por componente

Raspberry Pi 4 (2 GB)

Disponible en Chile a través de distribuidores oficiales como `raspberrypi.cl` y `mcielectronics.cl`. Se requiere el modelo Pi 4B con al menos 2 GB de RAM. Modelos anteriores (3B+ o inferiores) no tienen suficiente rendimiento de emulación, y modelos con menos RAM dejan sin margen al sistema.

Nota

Si la Pi 4 (2 GB) está sin stock, la Pi 5 (2 GB) sirve también y rinde mejor. La diferencia de precio suele ser de 10 % adicional. Cualquiera de las dos cumple los requisitos.

Fuente USB-C oficial

Crítica. La Pi 4 demanda 5 V a 3 A reales bajo carga. Cargadores de teléfono móvil, incluso los de marca, suelen entregar voltaje inestable, lo cual provoca *undervoltage warnings*, reinicios espontáneos y corrupción de la microSD. La fuente oficial es la única forma confiable de evitarlo.

MicroSD

32 GB es más que suficiente. Las especificaciones importantes son:

- **Clase 10:** garantiza 10 MB/s de escritura sostenida.
- **Endurance A1 o A2:** optimizado para aplicaciones (no solo grandes archivos como video).

Marcas recomendadas: SanDisk Endurance, Samsung Evo Plus.

Tip

Las microSD “baratas sin marca” fallan en cuestión de meses cuando un sistema operativo escribe constantemente. Para un *appliance* que vive encendido, vale la pena gastar un poco más.

Grandstream HT801

Analog Telephone Adapter (ATA) con un puerto FXS (*Foreign eXchange Subscriber*), es decir, un puerto que provee voltaje de línea (~48 V DC) y voltaje de timbre (~90 V AC) hacia un teléfono analógico, y por el otro lado se conecta por Ethernet a la red local y se registra como cliente SIP.

Características relevantes:

- Codecs G.711 (PCMU/PCMA), G.722, G.729, iLBC, OPUS
- DTMF in-band, RFC 2833 y SIP INFO
- Configurable vía web
- Función *Off-hook Auto-Dial*: marca automáticamente un número al descolgar

En Chile se consigue por MercadoLibre, distribuidores VoIP como Inttelec o Centralinet, o importación directa por Aliexpress (con riesgo de garantía).

Teléfono analógico

Cualquier teléfono fijo con teclado DTMF de los últimos cuarenta años funciona. **Los teléfonos de disco rotatorio no sirven** porque generan pulsos, no tonos. Modelos clásicos que cumplen el rol y aportan estética noventera:

- Panasonic KX-T series
- General Electric con cable rizado
- VTech inalámbrico (los primeros, con base grande)
- Cualquier teléfono “de hotel” Cetis o similar

3.2 Componentes opcionales

Componente	Razón	Precio CLP
TV CRT pequeño usado	Estética auténtica años 90	\$20.000–40.000
Switch Ethernet 5 puertos	Si el router está lejos del setup	\$10.000
Capturadora HDMI USB	Para grabar/transmitir partidas	\$25.000
Parlantes externos 3.5 mm	Si el TV tiene audio pobre	\$15.000
Etiquetas decorativas	Diseño tipo “Jugador 1” en el teléfono	\$3.000

Tabla 3.2: Componentes opcionales.

3.3 Plan de crecimiento: multijugador

El sistema base soporta un solo teléfono. Para escalar al modo televisión con múltiples jugadores simultáneos:

- Reemplazar el HT801 por un **HT802** (2 puertos FXS, ~\$55.000) o **HT814** (4 puertos FXS, ~\$110.000).
- Adquirir teléfonos adicionales.
- Reconfigurar Asterisk con ConfBridge y lógica de cola de turnos.

Este crecimiento se documenta en detalle en el capítulo 9.

3.4 Comparativa con alternativas

	Pi 4 (2 GB)	Pi 5 (2 GB)	Mini PC x86 usado
Precio aprox. (CLP)	\$85.000	\$95.000	\$100.000
Emulación de Hugo	Buena	Muy buena	Excelente (nativo)
Consumo eléctrico	5–7 W	8–12 W	15–25 W
Asterisk	Perfecto	Perfecto	Perfecto
Tamaño físico	Tarjeta de crédito	Tarjeta de crédito	Pequeño
HDMI a TV	Sí (micro)	Sí (micro)	Sí (estándar)
Estética appliance	Excelente	Excelente	Buena

Tabla 3.3: Comparativa de plataformas para Hugo Phone.

Para este proyecto se selecciona la **Pi 4 (2 GB)** por la combinación de precio, tamaño y eficiencia energética.

Fase 1: Preparación de la Raspberry Pi

4.1 Objetivo

Al final de esta fase, la Raspberry Pi debe estar:

- Booteando con Raspberry Pi OS Lite (64-bit).
- Accesible por SSH desde la Mac de trabajo.
- Con IP estática reservada en el router.
- Con Asterisk 22 instalado y respondiendo en su CLI (módulo `chan_pjsip`).
- Con los archivos de configuración del proyecto colocados en `/etc/asterisk/`.

4.2 Materiales

- Raspberry Pi 4 (2 GB)
- MicroSD 32 GB
- Fuente USB-C oficial
- Cable micro-HDMI
- Teclado USB (solo para el primer arranque)
- Televisor o monitor con HDMI
- Mac (u otra computadora) con lector de microSD

4.3 Paso 1: Flashear la microSD

Desde la Mac, descargar e instalar el *Raspberry Pi Imager* desde el sitio oficial [raspberrypi.com/software](https://www.raspberrypi.com/software). Conectar la microSD a la Mac (con adaptador USB si es necesario).

Al abrir el Imager configurar:

- **Device:** Raspberry Pi 4
- **Operating System:** Raspberry Pi OS Lite (64-bit)
- **Storage:** la microSD insertada

Antes de presionar *Write*, hacer clic en el ícono de engranaje (**Edit Settings**) para configurar:

- **Hostname:** hugo-pbx

- **Usuario:** beto (o el nombre que se prefiera), con contraseña robusta.
- **SSH:** habilitado, autenticación por contraseña o por clave pública.
- **WiFi:** SSID y password (solo si no se usará Ethernet).
- **Locale:** America/Santiago, teclado latam.

Confirmar y dejar que el Imager escriba (entre 5 y 15 minutos según la velocidad del lector).

4.4 Paso 2: Primer arranque

Insertar la microSD en la Pi, conectar HDMI, teclado USB y, finalmente, la fuente. El primer boot tarda entre 1 y 2 minutos porque expande automáticamente el sistema de archivos. Al terminar mostrará un *login prompt*.

Iniciar sesión con el usuario y contraseña configurados en el paso anterior.

4.5 Paso 3: Configuración de red

Lo deseable es asignar una IP estática reservada en el router para la dirección MAC de la Pi. Para obtener la MAC del puerto Ethernet:

```
1 ip link show eth0 | grep ether
```

La salida tendrá la forma `link/ether dc:a6:32:xx:xx:xx`. Anotar esa MAC, ingresar al panel de administración del router (habitualmente `http://192.168.1.1`) y reservar la IP `192.168.1.10` para esa MAC.

Reiniciar la Pi y verificar la IP:

```
1 ip addr show eth0
```

Tip

Si el router asigna IPs en otro rango (por ejemplo `192.168.0.x` o `10.0.0.x`), adaptar todas las direcciones del manual en consecuencia.

4.6 Paso 4: Conexión por SSH

Desde la Mac:

```
1 ssh beto@192.168.1.10
```

Si el router resuelve mDNS, también funciona:

```
1 ssh beto@hugo-pbx.local
```

A partir de este punto el teclado USB y el HDMI ya no son necesarios.

4.7 Paso 5: Actualización del sistema

```
1 sudo apt update
2 sudo apt upgrade -y
3 sudo reboot
```

Reconectar SSH después del reinicio.

4.8 Paso 6: Clonar el repositorio del proyecto

Copiar el directorio hugo-phone/ entregado en el repositorio del proyecto a la Pi. Si está en un servicio Git:

```
1 cd ~
2 git clone <URL-del-repositorio> hugo-phone
3 cd hugo-phone
```

Si los archivos están solo en la Mac:

```
1 # Desde la Mac
2 scp -r hugo-phone/ beto@hugo-pbx.local:~/
```

4.9 Paso 7: Ejecutar el instalador

```
1 cd ~/hugo-phone
2 sudo bash scripts/install-all.sh
```

El script realiza, en orden:

1. apt update && apt upgrade completos.
2. Instalación de Asterisk 22, DOSBox, build-essential, libudev-dev y demás dependencias.
3. Copia de los archivos pjsip.conf, extensions.conf y manager.conf a /etc/asterisk/ con permisos correctos.
4. Instalación de *rustup* en el usuario beto (descarga e instalación de la toolchain estable).
5. Compilación del bridge en modo **release**, con LTO y **strip**.
6. Creación de la regla udev para /dev/uinput y carga del módulo del kernel.
7. Instalación del servicio **systemd** hugo-bridge.service.
8. Despliegue de la configuración de DOSBox en /opt/hugo-phone/games/dosbox-hugo.conf (se invoca con **dosbox -conf**).

La fase más lenta es la compilación del bridge, que en una Pi 4 toma entre 5 y 8 minutos. Tiempo total estimado: 20–30 minutos según velocidad de red.

4.10 Paso 8: Configurar secretos

Las contraseñas iniciales son placeholders y deben reemplazarse antes de cualquier uso real:

```
1 sudo nano /etc/asterisk/pjsip.conf
2 sudo nano /etc/asterisk/manager.conf
3 sudo nano /opt/hugo-phone/bridge/config.toml
```

Reemplazar todas las ocurrencias de `cambiame-101`, `cambiame-102` y `cambiame-ami`.

Advertencia

La contraseña en `manager.conf` (sección `[hugo-bridge]`) y la del archivo `config.toml` (sección `[ami]`) deben ser **idénticas**. De lo contrario el bridge no podrá autenticarse en Asterisk.

Aplicar cambios reiniciando Asterisk:

```
1 sudo systemctl restart asterisk
```

4.11 Paso 9: Verificación

Ingresar a la consola interactiva de Asterisk:

```
1 sudo asterisk -rvvv
```

Dentro del CLI ejecutar:

```
hugo-pbx*CLI> pjsip show endpoints
```

Debe listar las extensiones 101 y 102 (esta última se usará en la siguiente fase) con estado `Not in use` (sin clientes registrados aún).

```
hugo-pbx*CLI> manager show settings
```

Debe indicar `Enabled: Yes` y `Port: 5038`.

```
hugo-pbx*CLI> dialplan show hugo-game
```

Debe mostrar la extensión 9000 con sus pasos de `Answer`, `Wait`, `Playback(beep)`, `Read` y `UserEvent`.

Si los tres comandos responden sin error, la **Fase 1 está completa**.

4.12 Resolución de problemas

4.12.1 Asterisk no inicia

```
1 sudo journalctl -u asterisk -n 80
```

Casi siempre se trata de un error de sintaxis en uno de los archivos `.conf` que el log indica con número de línea exacto.

4.12.2 La Pi se reinicia sin causa aparente o presenta lag aleatorio

Probablemente la fuente de poder es insuficiente. La Pi muestra un ícono de rayo amarillo en pantalla cuando detecta *undervoltage*. Cambiar a la fuente oficial 5 V/3 A.

4.12.3 SSH no responde

- Verificar que el cable Ethernet esté conectado o que el WiFi haya tomado correctamente la red. Conectar teclado + HDMI y ejecutar `ip addr`.
- Revisar que el servicio SSH esté corriendo: `sudo systemctl status ssh`.
- Confirmar que el firewall del router no esté bloqueando el tráfico LAN.

4.12.4 El instalador falla durante la compilación del bridge

- Reintentar manualmente como usuario beto:

```
1 cd /opt/hugo-phone/bridge
2 ~/.cargo/bin/cargo build --release
```

- Si falla por falta de memoria (poco probable en Pi 4 2 GB, pero posible), agregar *swap*:

```
1 sudo dphys-swapfile swapoff
2 sudo sed -i 's/CONF_SWAPSIZE=.* /CONF_SWAPSIZE=2048/' \
3   /etc/dphys-swapfile
4 sudo dphys-swapfile setup
5 sudo dphys-swapfile swapon
```

Fase 2: Prueba SIP con softphone

5.1 Objetivo

Validar la centralita Asterisk antes de tocar el HT801. Para ello se registra un *softphone* (Linphone) como extensión 102, se llama al número 9000 y se confirma que los tonos DTMF marcados desde la app llegan a Asterisk en forma de eventos AMI.

Este paso es importante porque permite verificar la parte de software del sistema sin depender aún del hardware analógico, simplificando el diagnóstico de problemas.

5.2 Paso 1: Instalar Linphone

Linphone es un cliente SIP gratuito y multiplataforma:

- **iOS:** App Store, buscar Linphone.
- **Android:** Google Play, buscar Linphone.
- **macOS:** linphone.org/technical-corner/linphone.

Instalar la aplicación en cualquiera de los dispositivos disponibles que esté en la misma red local que la Pi.

5.3 Paso 2: Crear cuenta SIP en Linphone

Abrir Linphone y entrar al asistente: Assistant → Use a SIP account (o Add Account, según versión).

Completar:

Campo	Valor
Username	102
Password	el secreto definido para la extensión 102 en <code>pjsip.conf</code>
Domain	192.168.1.10 (IP de la Pi)
Transport	UDP
Display Name	Test Hugo

Confirmar y verificar que el indicador de estado se ponga en verde con la leyenda **Registered** o **Connected**.

5.4 Paso 3: Verificar registro desde Asterisk

En la Pi:

```
1 sudo asterisk -rvvv
```

Dentro del CLI:

```
hugo-pbx*CLI> pjsip show endpoints
Endpoint: <Endpoint/CID.....> <State.....>
          <Channels.>
Contact:  <AOR/ContactUri.....> <Hash....> <Status>
          <RTT(ms)..>
=====

Endpoint: 101/101                                     Unavailable
          0 of inf
Endpoint: 102/102                                     Not in use
          0 of inf
          Contact: 102/sip:102@192.168.1.20:xxxx       abcdef0123
          Avail          3.000
```

La extensión 102 debe aparecer con un *Contact* (la IP del dispositivo donde corre Linphone) y estado Avail.

5.5 Paso 4: Llamada de prueba al 9000

Desde Linphone, marcar 9000 y presionar el botón de llamada.

Se deben observar dos cosas simultáneamente:

1. En el auricular del dispositivo: un *beep* corto, producto del paso `Playback(beep)` en el dialplan.
2. En la consola de Asterisk: la traza de ejecución del dialplan.

```
-- Executing [9000@hugo-game:1] NoOp("PJSIP/102-...", "Hugo Phone - ...")
-- Executing [9000@hugo-game:2] Answer("PJSIP/102-...", "")
-- Executing [9000@hugo-game:3] Wait("PJSIP/102-...", "1")
-- Executing [9000@hugo-game:4] Playback("PJSIP/102-...", "beep")
-- <PJSIP/102-...> Playing 'beep.gsm' (language 'es')
-- Executing [9000@hugo-game:5] Read("PJSIP/102-...", "TECLA,,1,,3600")
```

5.6 Paso 5: Probar DTMF

Mientras la llamada sigue activa, en Linphone abrir el teclado de tonos y marcar el dígito **2**.

En la consola de Asterisk se debe ver:

```
== Manager 'hugo-bridge' logged on from 127.0.0.1
== UserEvent: HugoKey
   Channel: PJSIP/102-00000001
   Caller: 102
```

```
Key: 2
-- Executing [9000@hugo-game:6] GotoIf(...)
-- Executing [9000@hugo-game:5] Read("PJSIP/102-...", "TECLA,,1,,3600")
```

Repetir con **4**, **6**, **8** y **5**. Cada uno debe disparar su propio `UserEvent`.

Si los eventos aparecen, la **Fase 2 está completa**: la centralita captura DTMF correctamente y el bridge podrá leerlos.

5.7 Resolución de problemas

5.7.1 Linphone no se registra

- Verificar que la Pi sea alcanzable desde el dispositivo: `ping 192.168.1.10`.
- Confirmar que el puerto UDP 5060 no esté bloqueado por firewall en la Pi (Raspberry Pi OS Lite no trae firewall activado por defecto).
- Revisar logs de Asterisk en busca de intentos fallidos:

```
1 sudo asterisk -rvvv
2 hugo-pbx*CLI> pjsip set logger on
```

5.7.2 La llamada al 9000 se corta inmediatamente

- Verificar que la extensión 102 esté asignada al contexto `hugo-game` en `pjsip.conf`.
- Ejecutar `dialplan show hugo-game` en el CLI y confirmar que la extensión 9000 está presente.

5.7.3 Conecta pero no se escucha el beep

- Problema típico de NAT o de codecs incompatibles. Verificar que `allow=ulaw` y `allow=alaw` estén en la plantilla `endpoint-template` de `pjsip.conf` y que Linphone también soporte estos codecs en su configuración avanzada.
- Revisar el volumen del dispositivo.

5.7.4 No aparecen UserEvent al marcar DTMF

- Linphone puede enviar DTMF como tonos *inband* en lugar de RFC 2833. En `Settings` → `Audio` → `DTMF mode` forzar el método `RFC 2833` o `SIP INFO`.
- Confirmar que la plantilla `endpoint-template` en `pjsip.conf` tiene la línea `dtmf_mode=rfc4733` (equivalente PJSIP de RFC 2833).

5.8 Cierre de fase

En este punto está validado:

- Asterisk acepta clientes SIP y los registra correctamente.

- El dialplan responde llamadas y reproduce audio.
- La función `Read()` captura DTMF en variable.
- El `UserEvent` se emite y es visible en AML.

Esto es todo lo que el bridge necesita del lado de Asterisk. La siguiente fase prepara DOSBox y Hugo para que el bridge tenga algo donde inyectar las teclas.

Fase 3: DOSBox y Hugo

6.1 Objetivo

Tener Hugo ejecutándose en la Pi, mostrado en pantalla por HDMI y jugable con teclado USB. Esta es la última pieza individual antes de unirlo todo en la Fase 4.

6.2 Estado legal del juego

Hugo es *abandonware*: software comercial cuyo soporte por parte del editor original cesó hace décadas y cuya distribución pasiva es ampliamente tolerada aunque no formalmente autorizada. Sitios como myabandonware.com y abandonwaregames.net alojan copias descargables. La situación legal varía por jurisdicción, pero el uso doméstico y educativo no ha sido objeto de litigios.

6.3 Paso 1: Conseguir Hugo

La saga tiene varias entregas y los formatos varían:

- **Hugo 1 / 2 / 3** (1990–1992) vienen como ZIP que al descomprimir deja un HUGO.EXE listo para ejecutar.
- **Hugo 4 y posteriores** suelen distribuirse como imagen ISO de CD-ROM con INSTALL.EXE, INSTALLW.EXE y HUGO.EXE. Requieren correr el instalador dentro de DOSBox la primera vez.

Una vez descargado, transferir el archivo a la Pi:

```
1 # Desde la Mac, ZIP (Hugo 1/2/3):
2 scp ~/Downloads/Hugo*.zip beto@hugo-pbx.local:/tmp/
3
4 # Desde la Mac, ISO (Hugo 4 y otros CD-ROM):
5 scp ~/Downloads/hugo4.iso \
6     beto@hugo-pbx.local:/opt/hugo-phone/games/
```

6.3.1 Caso A: ZIP con HUGO.EXE suelto

```
1 sudo mkdir -p /opt/hugo-phone/games/HUGO
2 sudo unzip /tmp/Hugo*.zip -d /opt/hugo-phone/games/HUGO
3 sudo chown -R beto:beto /opt/hugo-phone/games/HUGO
4 ls /opt/hugo-phone/games/HUGO/
```

Debería listar HUGO.EXE (o un nombre similar). Pasar al Paso 4.

6.3.2 Caso B: ISO de CD-ROM (Hugo 4)

El ISO permanece como archivo. DOSBox lo monta como unidad D: con `imgmount`. La primera vez hay que correr el instalador (`INSTALL.EXE`, no `INSTALLW.EXE` que es para Windows) para que copie archivos a `C:\HUGO\` y configure el audio.

6.4 Paso 2: Instalar DOSBox

DOSBox (clásico) está en los repos oficiales de Raspberry Pi OS / Debian Trixie:

```
1 sudo apt update
2 sudo apt install -y dosbox xinit openbox xserver-xorg
3 which dosbox
```

Nota

El paquete `dosbox-staging` no está disponible en el repositorio estándar de la Pi al momento de escribir este manual. Si en el futuro lo está, las opciones del `dosbox-hugo.conf` requerirían adaptarse a la sintaxis de *Staging*.

6.5 Paso 3: Instalador del juego (sólo Caso B / ISO)

Conectar HDMI + teclado USB a la Pi y loguearse en la TTY local (no por SSH). DOSBox necesita pantalla.

Lanzar el instalador con el ISO montado como D::

```
1 startx /usr/bin/dosbox -- \
2   -c "mount c /opt/hugo-phone/games" \
3   -c "imgmount d /opt/hugo-phone/games/hugo4.iso -t iso" \
4   -c "d:" \
5   -c "INSTALL.EXE"
```

Dentro del instalador:

- **Destino:** `C:\HUGO` (o lo que sugiera por defecto).
- **Sound card:** Sound Blaster 16, base 220, IRQ 7, DMA 1. Coincide con la sección `[sblaster]` del `dosbox-hugo.conf`.
- Aceptar y salir del instalador.

Verificar que los archivos quedaron copiados:

```
1 ls /opt/hugo-phone/games/HUGO/
```

6.6 Paso 4: Verificar la configuración de DOSBox

El script de instalación copió `dosbox-hugo.conf` a `/opt/hugo-phone/games/`. Verificar:

```
1 head -30 /opt/hugo-phone/games/dosbox-hugo.conf
```

Si el ejecutable del juego no se llama exactamente HUGO.EXE o vive en una subcarpeta distinta, editar la sección [autoexec]:

```
1 nano /opt/hugo-phone/games/dosbox-hugo.conf
```

Sección por defecto (Caso B, Hugo 4 ISO con CD presente):

```
[autoexec]
mount c /opt/hugo-phone/games
imgmount d /opt/hugo-phone/games/hugo4.iso -t iso
c:
cd HUGO
HUGO.EXE
exit
```

Para Caso A (HUGO.EXE suelto, sin ISO), comentar la línea imgmount.

6.7 Paso 5: Lanzar Hugo

DOSBox del paquete de Debian usa SDL 1.2 y requiere un servidor X (no soporta KMSDRM directo). El camino más simple es startx + DOSBox dentro:

```
1 startx /usr/bin/dosbox -conf /opt/hugo-phone/games/dosbox-hugo.conf
```

Esto inicia un X11 mínimo, ejecuta DOSBox dentro y, al cerrar DOSBox, X termina y vuelve a la TTY.

Nota

startx sólo funciona desde una TTY local (no por SSH). Para pruebas remotas se puede usar SSH con X forwarding (ssh -Y), pero el rendimiento es muy inferior y no se recomienda para jugabilidad real.

6.8 Paso 6: Arranque automático del juego

Configurar la Pi para que al encender entre directamente a Hugo, sin intervención del usuario.

6.8.1 Habilitar autologin en TTY1

```
1 sudo raspi-config
```

Navegar a: System Options → Boot / Auto Login → Console Autologin. Guardar y salir.

6.8.2 Ejecutar DOSBox al iniciar sesión

Editar el .bash_profile del usuario beto:

```
1 nano ~/.bash_profile
```

Agregar al final:

```
if [ -z "$DISPLAY" ] && [ "$(tty)" = "/dev/tty1" ]; then
    startx /usr/bin/dosbox \
        -conf /opt/hugo-phone/games/dosbox-hugo.conf
fi
```

La condición evita que DOSBox se ejecute si se ingresa al sistema por SSH o desde una TTY distinta de la 1.

Al reiniciar, la secuencia debe ser:

1. Boot del kernel
2. Autologin como beto en tty1
3. startx arranca X11
4. DOSBox arranca, ejecuta [autoexec] y aparece Hugo

6.9 Paso 7: Prueba de jugabilidad

Con un teclado USB conectado a la Pi y Hugo cargado, probar:

- Flechas direccionales → movimiento del personaje
- Barra espaciadora → acción
- Esc → menú/salir

Si Hugo se siente excesivamente rápido o lento, ajustar la línea `cycles` en `dosbox-hugo.conf`:

```
1 [cpu]
2 cycles = fixed 8000    # Aumentar si va lento, disminuir si va rapido
```

Hugo original requería aproximadamente un 386SX a 25 MHz. Empezar con `cycles = fixed 8000` y calibrar.

6.10 Paso 8: Salir limpiamente

Dos pulsaciones de Esc salen al menú de Hugo. La opción *Quit* regresa al prompt de DOSBox; el comando `exit` del bloque [autoexec] cierra DOSBox automáticamente.

Para salir de emergencia (por ejemplo si el juego se cuelga): `Ctrl+F9` es el *kill switch* de DOSBox.

6.11 Resolución de problemas

6.11.1 startx no encuentra display o falla

Confirmar que están los paquetes del servidor X:

```
1 sudo apt install -y xinit openbox xserver-xorg
```

Y que estás logueado en una TTY local (no SSH). Verificar con `tty` (debe responder `/dev/tty1` o similar, no `/dev/pts/*`).

6.11.2 Hugo se cuelga en la intro o las cinemáticas

DOSBox en arquitectura ARM ocasionalmente tiene problemas con la decodificación de animaciones VGA. Pulsar cualquier tecla para saltar la intro suele resolverlo. Si persiste, bajar la calidad del *output*:

```
1 [sdl]
2 output = surface
```

6.11.3 El audio se entrecorta

Aumentar el buffer del mezclador:

```
1 [mixer]
2 prebuffer = 50
3 blocksize = 2048
```

Si aún hay cortes, bajar la calidad: `rate = 22050`.

6.11.4 El juego pide CD pero el ISO está montado

Verificar que el `imgmount` no falló silenciosamente:

```
1 ls -la /opt/hugo-phone/games/hugo4.iso
```

Debe existir y ser legible por el usuario `beto`. Si no, ajustar permisos:

```
1 sudo chown beto:beto /opt/hugo-phone/games/hugo4.iso
```

6.11.5 Hugo arranca pero no responde al teclado

- Verificar que `core = dynamic` esté presente en la sección `[cpu]`.
- Algunos títulos requieren `cputype = 386` en lugar de `auto`.
- Asegurarse de que la ventana de DOSBox tiene el foco (con `autolock = true` se captura el teclado al hacer clic).

6.12 Cierre de fase

En este punto el juego corre con teclado USB. La siguiente fase reemplaza ese teclado USB por uno virtual controlado desde el teléfono.

Fase 4: El bridge en Rust

7.1 Objetivo

Activar el bridge `hugo-bridge`, que conecta los eventos AMI de Asterisk con el subsistema `uinput` del kernel, traduciendo DTMF a pulsaciones de teclado virtual.

Esta es la fase con mayor contenido de software del proyecto. Se aprovecha para documentar en detalle la arquitectura interna del bridge.

7.2 Estructura del código

```
bridge/
|-- Cargo.toml           # Manifest del crate
|-- config.toml          # Configuración en runtime
|-- hugo-bridge.service  # Unit systemd
`-- src/
    |-- main.rs          # Entrada, tokio runtime
    |-- config.rs        # Parseo TOML con serde
    |-- ami.rs           # Cliente AMI minimal
    |-- keyboard.rs      # Teclado virtual uinput
    `-- watcher.rs       # File watcher (hot-reload)
```

7.3 Dependencias y filosofía de diseño

El bridge fue diseñado para minimizar dependencias externas. Las únicas *crates* en `Cargo.toml` son:

- `tokio`: runtime asincrónico (con *features* específicas, no `full`).
- `serde` + `toml`: parseo del archivo de configuración.
- `tracing` + `tracing-subscriber`: logging estructurado.
- `clap`: parseo de argumentos de línea de comandos.
- `uinput`: *wrapper* sobre la API `ioctl` de `/dev/uinput`.
- `notify`: watcher de archivos para hot-reload.

Conspicuamente ausente: ningún cliente AMI existente. El protocolo es lo suficientemente simple como para implementarlo a mano en cerca de 150 líneas, evitando arrastrar dependencias adicionales y manteniendo el control sobre cada byte que pasa por el socket.

7.4 El protocolo AMI

Asterisk Manager Interface es un protocolo de texto plano sobre TCP. Su especificación informal es:

- Conexión TCP a 127.0.0.1:5038.
- El servidor envía un banner: Asterisk Call Manager/X.Y.Z\r\n.
- Los mensajes (en ambas direcciones) consisten en pares Clave: Valor separados por \r\n, terminados por una línea en blanco (\r\n\r\n).
- El cliente debe autenticarse con un mensaje Action: Login y campos Username y Secret.
- Posteriormente, el servidor envía eventos asincrónicos cuando ocurren.

Un ejemplo de mensaje real:

```
Event: UserEvent
Privilege: user,all
UserEvent: HugoKey
Channel: PJSIP/102-00000001
Caller: 102
Key: 2
Uniqueid: 1715641234.1
(linea en blanco)
```

7.5 Anatomía del módulo ami.rs

El cliente AMI se implementa en cuatro funciones:

7.5.1 run: bucle principal con reconexión

```
1 pub async fn run(mut cfg_rx: watch::Receiver<Arc<Config>>) -> Result<()> {
2     let initial_cfg = cfg_rx.borrow().clone();
3     let mut keyboard = VirtualKeyboard::new(
4         &initial_cfg.keymap,
5         initial_cfg.bridge.press_duration_ms,
6     )?;
7
8     let mut backoff_secs = 1u64;
9     loop {
10        let cfg = cfg_rx.borrow().clone();
11        match connect_and_loop(&cfg, &mut keyboard, &mut cfg_rx).await {
12            Ok(()) => {
13                backoff_secs = 1;
14            }
15            Err(e) => {
16                error!("Error AMI: {}. Reintentando en {}s...", e,
17                    backoff_secs);
18                sleep(Duration::from_secs(backoff_secs)).await;
19                backoff_secs = (backoff_secs * 2).min(30);
20            }
21        }
22    }
23 }
```

```

19         }
20     }
21 }
22 }

```

Tres decisiones clave:

- **El teclado se crea una sola vez**, fuera del bucle de conexión. Recrearlo entre reconexiones provoca un breve período donde el dispositivo `/dev/input/eventXX` desaparece, lo cual SDL2 maneja mal.
- **Backoff exponencial** entre reintentos (1, 2, 4, 8, 16, 30, 30, ...), saturado en 30 segundos para no esperar minutos si Asterisk se reinicia.
- **El *receiver* de config se pasa por referencia mutable** para que la función interna pueda detectar cambios de configuración mientras está conectada.

7.5.2 connect_and_loop: conexión, login y bucle de eventos

Tras conectar el socket TCP, leer el banner y enviar el Action: Login, el bucle principal usa `tokio::select!` para multiplexar dos fuentes de eventos:

```

1 loop {
2     tokio::select! {
3         msg = read_message(&mut reader) => {
4             let msg = msg?;
5             handle_event(&msg, &event_name, &key_field, keyboard).await;
6         }
7         changed = cfg_rx.changed() => {
8             if changed.is_err() { return Ok(()); }
9             let new_cfg = cfg_rx.borrow().clone();
10            keyboard.update_keymap(&new_cfg.keymap);
11            keyboard.set_press_duration(new_cfg.bridge.press_duration_ms);
12            if new_cfg.ami != cfg.ami {
13                return Ok(()); // fuerza reconexion
14            }
15        }
16    }
17 }

```

El `select!` bloquea hasta que ocurra una de las dos cosas. Si llega un mensaje, se procesa. Si cambió la configuración (recargada en caliente por el watcher), se aplica al teclado y, si los parámetros de AMI cambiaron, se fuerza una reconexión retornando `Ok()` hacia el bucle externo.

7.5.3 handle_event: filtrado y acción

```

1 async fn handle_event(
2     msg: &HashMap<String, String>,
3     event_name: &str,
4     key_field: &str,

```



```

5     keyboard: &mut VirtualKeyboard,
6 ) {
7     if msg.get("Event").map(String::as_str) != Some("UserEvent") { return; }
8     if msg.get("UserEvent").map(String::as_str) != Some(event_name) { return;
9         }
10
11     let Some(dtmf) = msg.get(key_field) else { return };
12     let caller = msg.get("Caller").map(String::as_str).unwrap_or("?");
13
14     info!("DTMF {} de {}", dtmf, caller);
15     if let Err(e) = keyboard.press_dtmf(dtmf).await {
16         error!("Fallo al pulsar tecla: {}", e);
17     }
18 }

```

Tres filtros sucesivos descartan eventos AMI que no son del interés del bridge. Solo los UserEvent de tipo HugoKey (configurable) pasan a la inyección de tecla.

7.5.4 read_message: parser del wire format

```

1  async fn read_message<R: tokio::io::AsyncBufRead + Unpin>(
2      reader: &mut R,
3  ) -> std::result::Result<HashMap<String, String>, BoxError> {
4      let mut map = HashMap::new();
5      let mut line = String::new();
6      loop {
7          line.clear();
8          let n = reader.read_line(&mut line).await?;
9          if n == 0 {
10             return Err("Conexion AMI cerrada por el servidor".into());
11         }
12         let trimmed = line.trim_end_matches(['\r', '\n']);
13         if trimmed.is_empty() {
14             if map.is_empty() { continue; }
15             return Ok(map);
16         }
17         if let Some((k, v)) = trimmed.split_once(':') {
18             map.insert(k.trim().to_string(), v.trim().to_string());
19         }
20     }
21 }

```

Lee línea por línea hasta encontrar una línea vacía, momento en el cual considera el mensaje completo. Líneas mal formadas (sin :) se descartan silenciosamente, comportamiento típico de parsers tolerantes en protocolos legacy.

7.6 El módulo keyboard.rs

El subsistema uinput del kernel Linux permite a un proceso en *user space* crear dispositivos de entrada virtuales. El kernel los expone como cualquier teclado físico bajo `/dev/input/eventXX`

y aplicaciones como DOSBox no pueden distinguirlos de hardware real.

7.6.1 Creación del teclado virtual

```
1 let device = uinput::default()?
2   .name("hugo-phone-virtual-keyboard")?
3   .event(uinput::event::Keyboard::All)?
4   .create()?;
```

La constante `Keyboard::All` registra todas las teclas conocidas por el kernel. Esto es importante para el hot-reload: si solo se registraran las teclas presentes en el *keymap* inicial, al cambiar el mapeo a una tecla nueva habría que destruir y recrear el dispositivo, lo cual interrumpe el flujo.

7.6.2 Inyección de pulsación

```
1 pub async fn press_dtmf(&mut self, dtmf: &str) -> Result<()> {
2     let Some(key) = self.keymap.get(dtmf) else {
3         warn!("DTMF {:?} sin mapeo", dtmf);
4         return Ok(());
5     };
6     self.device.press(key)?;
7     self.device.synchronize()?;
8     sleep(self.press_duration).await;
9     self.device.release(key)?;
10    self.device.synchronize()?;
11    Ok(())
12 }
```

Cada pulsación sigue el ciclo:

1. Buscar la tecla simbólica correspondiente al DTMF recibido.
2. Enviar evento `press`.
3. Llamar `synchronize()` para que el kernel emita un `SYN_REPORT` y propague el evento a los consumidores.
4. Esperar `press_duration` (típicamente 80 ms).
5. Enviar evento `release` y volver a sincronizar.

La pausa entre *press* y *release* es importante: si fuera demasiado corta, juegos como Hugo no detectan la pulsación; si es demasiado larga, el motor del juego dispara la acción en repetición continua.

7.7 El módulo `watcher.rs`

El *hot-reload* de configuración permite ajustar el *keymap* y la duración de pulsación sin reiniciar el servicio. Es particularmente útil durante la calibración inicial.

```
1 let mut watcher = notify::recommended_watcher(move |res: notify::Result<Event  
  >| {  
2     match res {  
3         Ok(evt) => { let _ = tx.send(evt); }  
4         Err(e) => error!("Watcher error: {}", e),  
5     }  
6 })?;  
7 watcher.watch(parent, RecursiveMode::NonRecursive)?;
```

Dos detalles importantes:

- Se observa el **directorio padre**, no el archivo en sí. Muchos editores guardan un archivo escribiendo a un temporal y renombrándolo encima del original (*atomic save*), lo cual no dispara un evento *modify* sobre el archivo original sino un *create* en el directorio.
- Se aplica un *debounce* de 500 ms para evitar recargas múltiples si el editor escribe en varios pasos.

7.8 Verificación práctica

Probar el bridge manualmente:

```
1 cd /opt/hugo-phone/bridge  
2 sudo ./hugo-bridge --config config.toml
```

Salida esperada:

```
INFO Config cargado desde "/opt/hugo-phone/bridge/config.toml"  
INFO Teclado virtual creado  
INFO Watching ".../config.toml" para hot-reload  
INFO Conectando a AMI en 127.0.0.1:5038 ...  
INFO AMI banner: Asterisk Call Manager/9.0.0  
INFO Autenticado en AMI como hugo-bridge
```

Verificar la existencia del teclado virtual:

```
1 sudo apt install -y evtest  
2 sudo evtest
```

En la lista debe aparecer `hugo-phone-virtual-keyboard`. Seleccionar su número y luego marcar DTMF en Linphone: el `evtest` debe mostrar los eventos de pulsación correspondientes.

7.9 El flujo completo con softphone

Con el bridge corriendo, DOSBox mostrando Hugo en la TV y Linphone registrado:

1. Llamar al 9000 desde Linphone.
2. Escuchar el beep en el auricular.

3. Marcar 2. El personaje debe saltar/subir.

4. Probar 4, 6, 8, 5.

En los logs del bridge:

```
INFO DTMF 2 de 102
INFO DTMF 4 de 102
```

7.10 Calibración

El parámetro más sensible es `press_duration_ms` en `config.toml`:

```
1 [bridge]
2 press_duration_ms = 80
```

Síntomas y ajustes:

Síntoma	Ajuste
Una pulsación = varias acciones	Bajar a 40–50 ms
La pulsación no registra	Subir a 120–150 ms
El movimiento es entrecortado	Subir a 100 ms

Cualquier cambio se aplica al guardar (hot-reload), sin reiniciar el servicio.

7.11 Activar como servicio

Una vez validado manualmente:

```
1 sudo systemctl start hugo-bridge
2 sudo systemctl status hugo-bridge
3 sudo systemctl enable hugo-bridge
```

Logs en vivo:

```
1 sudo journalctl -u hugo-bridge -f
```

7.12 Resolución de problemas

7.12.1 Permission denied sobre /dev/uinput

- El servicio corre como root, no debería ocurrir.
- Si se está ejecutando manualmente, usar `sudo`.
- Verificar la regla udev en `/etc/udev/rules.d/99-hugo-uinput.rules`.

7.12.2 El bridge se conecta pero no recibe eventos

- Confirmar que Linphone está llamando al 9000 y no a otra extensión.
- En la consola de Asterisk, ¿aparece el `UserEvent: HugoKey` al marcar DTMF? Si no, el problema es en el dialplan (Fase 2).

7.12.3 Los DTMF llegan al bridge pero Hugo no responde

- DOSBox requiere tener el foco de ventana. Si se lanzó vía SSH con X-forwarding, lo pierde.
- Probar la inyección contra otra aplicación: con el bridge corriendo, abrir `nano` en la TTY de DOSBox y verificar que el cursor se mueva al marcar DTMF.

7.12.4 Cargo no encuentra dependencias durante la compilación

- Verificar conexión a Internet: `ping crates.io`.
- Si hay restricciones de DNS, configurar:

```
1 echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf
```

7.13 Cierre de fase

Con esta fase, todo el flujo software está validado. Solo resta reemplazar el softphone Linphone por un teléfono analógico real conectado al HT801.

Fase 5: Conectar el teléfono real

8.1 Objetivo

Reemplazar el softphone por un teléfono analógico DTMF físico, conectado a la centralita a través del adaptador Grandstream HT801. Al terminar esta fase, descolgar el teléfono debe llevar directamente al juego, sin marcar ningún número.

8.2 Conceptos previos

8.2.1 Qué es un puerto FXS

Un puerto *FXS* (*Foreign eXchange Subscriber*) es el tipo de puerto que provee la compañía telefónica al usuario doméstico: entrega voltaje continuo de aproximadamente 48 V para alimentar el teléfono, voltaje alterno de unos 90 V para hacerlo sonar, y mediante el *loop current* detecta cuándo se descuelga o cuelga el aparato.

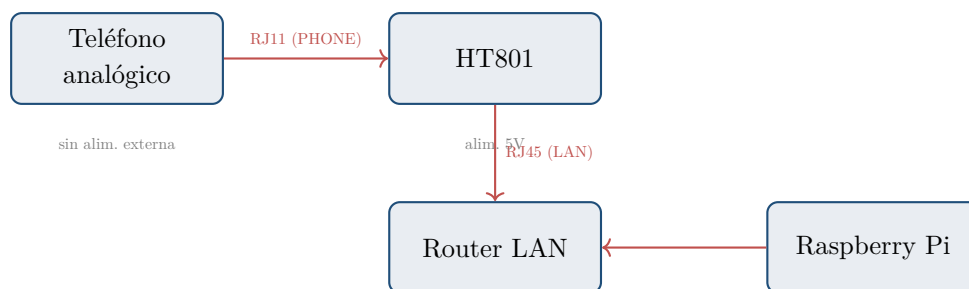
Un puerto *FXO* (*Foreign eXchange Office*) es el opuesto: se conecta hacia una central telefónica y se comporta como un teléfono.

El HT801 expone un puerto FXS por su conector RJ11, lo cual permite enchufar cualquier teléfono analógico como si la central fuera la compañía telefónica.

8.2.2 Auto-dial al descolgar

La función *Off-hook Auto-Dial* del HT801 permite configurar un número que se marcará automáticamente al descolgar el teléfono. Esta es la clave para replicar la experiencia Hugo: el usuario descuelga e inmediatamente está jugando, sin necesidad de marcar nada.

8.3 Conexiones físicas



- El cable RJ11 del teléfono va al puerto rotulado **PHONE** (o **FXS**) del HT801. *No* al puerto LAN.
- El HT801 va por Ethernet al mismo switch/router donde está la Pi.

- La fuente del HT801 (5 V) se conecta al puerto correspondiente.
- El teléfono analógico no necesita alimentación externa: la línea le provee voltaje.

8.4 Paso 1: Descubrir la IP del HT801

El HT801 obtiene IP por DHCP automáticamente. Para conocerla hay dos métodos:

8.4.1 Método A: IVR del propio HT801

1. Descolgar el teléfono conectado al HT801.
2. Marcar *** (tres asteriscos).
3. Una voz en inglés ofrece un menú de opciones de diagnóstico.
4. Marcar 02 para escuchar la IP del dispositivo.

8.4.2 Método B: panel del router

Entrar al panel de administración del router y buscar en la lista de clientes DHCP un dispositivo nuevo con nombre similar a GS_HT801 o HT801-xxxx.

Una vez identificada la IP (supongamos 192.168.1.11), **reservar** esa IP para la MAC del HT801 en el router, igual que se hizo con la Pi.

8.5 Paso 2: Acceder al panel web del HT801

Desde la Mac, abrir en el navegador:

```
http://192.168.1.11
```

Credenciales:

- **Usuario:** admin
- **Contraseña:** admin (modelos antiguos) o la contraseña impresa en la etiqueta inferior del dispositivo (modelos recientes con contraseña única por unidad).

Advertencia

Cambiar la contraseña por defecto antes de hacer cualquier otra cosa: Advanced Settings → Admin Password.

8.6 Paso 3: Configurar la cuenta SIP

Navegar a la pestaña FXS Port (o Profile 1 en algunos firmwares).

En la sección de codecs:

- Choice 1: PCMU (G.711 μ -law)

Campo	Valor
Account Active	Yes
Primary SIP Server	192.168.1.10 (IP de la Pi)
SIP User ID	101
Authenticate ID	101
Authenticate Password	el secreto de 101 en pjsip.conf
Name	Telefono Hugo 1
DTMF Payload Type	101
Preferred DTMF method	RFC2833 (priority 1)
SIP Registration	Yes
Outgoing call without registration	No
NAT Traversal	No

Tabla 8.1: Configuración SIP del HT801.

- Choice 2: PCMA (G.711 A-law)
- Resto: None

8.7 Paso 4: Configurar Auto-Dial

En la misma pestaña, buscar el campo `Offhook Auto-Dial` y configurarlo en:

```
9000
```

Esto provoca que al descolgar el teléfono, el HT801 marque automáticamente al 9000, que es la extensión que el dialplan asocia con el juego.

Hacer clic en `Update` y luego en `Apply`. El HT801 reinicia su módulo SIP y queda registrado al cabo de unos 10 segundos.

8.8 Paso 5: Verificar registro

En la Pi:

```
1 sudo asterisk -rvvv
```

```
hugo-pbx*CLI> pjsip show contacts
```

```
Contact: <Aor/ContactUri.....> <Hash....> <Status> <
RTT(ms) ..>
```

```
=====
Contact: 101/sip:101@192.168.1.11:5060          abcdef0123 Avail
5.000
```

La extensión 101 debe aparecer con la IP del HT801 y estado `Avail`. Si dice `Unavail` o el contacto no aparece, verificar:

- Coincidencia de contraseñas entre HT801 y `pjsip.conf`.

- Que la IP de *Primary SIP Server* apunte realmente a la Pi.
- Los logs detallados:

```
1 hugo-pbx*CLI> pjsip set logger on
```

8.9 Paso 6: La prueba final

Todas las piezas están conectadas. La secuencia:

1. Confirmar que DOSBox muestra Hugo en la TV.
2. Confirmar que el servicio bridge está activo: `systemctl status hugo-bridge`.
3. **Descolgar el teléfono.**
4. En el auricular debe escucharse el beep del dialplan (después de un breve momento mientras el HT801 marca 9000 internamente).
5. Marcar **2**. Hugo salta.
6. Marcar **4, 6, 8, 5** y jugar normalmente.
7. Al colgar el teléfono, la llamada finaliza limpiamente.

Si todo funciona, el sistema está completo en su configuración mínima. El proyecto Hugo Phone está vivo.

8.10 Resolución de problemas

8.10.1 El teléfono no da tono al descolgar

El HT801 no está dando línea. Causas posibles:

- LED del puerto FXS en rojo: error de hardware o configuración. Hacer un *factory reset* (botón Reset físico presionado 7 segundos).
- Cable RJ11 en mal estado o invertido (probar otro).
- Teléfono dañado (probar con otro teléfono).

8.10.2 Hay tono pero al descolgar no marca el 9000

Offhook Auto-Dial no está bien configurado. Verificar que sea exactamente 9000 sin espacios, sin asteriscos, sin numerales.

8.10.3 Conecta pero no se escucha el beep

Problema de codecs:

- Asegurar que el HT801 prioriza PCMU y PCMA.

- En el HT801, sección *Audio Settings*, subir las ganancias *Tx* y *Rx* si el volumen es muy bajo.

8.10.4 Los DTMF no producen acción en Hugo

- Verificar el método DTMF en el HT801: debe ser RFC2833.
- En `sudo asterisk -rvvv` debe aparecer el `UserEvent: HugoKey` al pulsar dígitos.
- Algunos HT801 antiguos fallan con RFC 2833 y requieren cambiar a SIP INFO. Probar ese método si el anterior no funciona.

8.10.5 La llamada se cae sola después de algunos minutos

El dialplan tiene un timeout de 3600 segundos (1 hora) en `Read()`. Si la llamada cae antes, probablemente un *session timer* está activo. Verificar en `pjsip.conf` que la plantilla `endpoint-template` tenga `timers=no` (equivalente PJSIP de `session-timers=refuse`).

Fase 6: Modo multijugador (futuro)

9.1 Visión

La configuración descrita hasta el capítulo anterior soporta un solo teléfono. La experiencia original de televisión, sin embargo, contemplaba múltiples jugadores llamando simultáneamente, escuchando el audio del juego mientras esperaban, y recibiendo turnos rotativos de 30 a 60 segundos.

Esta fase describe el diseño para escalar el sistema a esa experiencia completa. **No se documenta como un paso a paso ejecutable** (eso es trabajo de una versión 2.0 del proyecto), sino como una hoja de ruta arquitectónica.

9.2 Cambios de hardware

- Reemplazar el **HT801** (1 puerto FXS) por:
 - **HT802**: 2 puertos FXS. Costo aprox. \$55.000 CLP.
 - **HT814**: 4 puertos FXS. Costo aprox. \$110.000 CLP.
 - **Cisco SPA8000** (usado): 8 puertos FXS. Costo aprox. \$80.000–120.000 CLP.
- Adquirir tantos teléfonos analógicos adicionales como puertos FXS se utilicen.
- Opcionalmente, una capturadora HDMI USB para transmitir o grabar las partidas.

9.3 Cambios en Asterisk

9.3.1 pjsip.conf: nuevas extensiones

Agregar extensiones 102, 103, 104 según número de puertos FXS, replicando el bloque de `endpoint/auth/aor` que usa la 101 (heredando `endpoint-template`).

9.3.2 confbridge.conf: salas de conferencia

Crear el archivo `/etc/asterisk/confbridge.conf` con perfiles para dos tipos de participantes:

```
1 [default_bridge]
2 type=bridge
3 max_members=10
4 mixing_interval=20
5
6 [player_active]
7 type=user
```

```

8 admin=no
9 marked=yes
10 quiet=yes
11
12 [player_waiting]
13 type=user
14 admin=no
15 marked=no
16 quiet=yes
17 music_on_hold_when_empty=no

```

9.3.3 extensions.conf: lógica de cola

Modificar la extensión 9000 para que el primer participante entre como *player_active* y los siguientes como *player_waiting*:

```

1 [hugo-game]
2 exten => 9000,1,Answer()
3 same => n,Set(POSITION=${CONFBRIDGE_INFO(parties,hugo-room)})
4 same => n,GotoIf($[${POSITION} = 0]?active:waiting)
5 same => n(active),ConfBridge(hugo-room,default_bridge,player_active)
6 same => n(waiting),Playback(esperando-turno)
7 same => n,ConfBridge(hugo-room,default_bridge,player_waiting)

```

9.4 Lógica de turnos

Se requiere un componente adicional (puede ser otro proceso Rust o un módulo dentro del bridge existente) que cada 60 segundos:

1. Identifique al *player_active* actual mediante el evento `ConfbridgeList` o consultas `ConfbridgeList` sobre AMI.
2. Lo degrade a *waiting* mediante `ConfbridgeMute` y reconfiguración de perfil.
3. Promueva al siguiente jugador en cola (FIFO) a *active*.
4. Anuncie el cambio de turno reproduciendo un mensaje pregrabado a todos.

Solo el jugador activo debe poder controlar al personaje. Esto se logra haciendo que el bridge filtre los `UserEvent` por el campo `Channel` y solo acepte los del canal activo.

9.5 Audio del juego en la conferencia

Para que los jugadores en espera escuchen Hugo, hay que enrutar el audio de DOSBox hacia la sala de conferencia.

9.5.1 Camino A: ALSA loopback

```

1 sudo modprobe snd-aloop

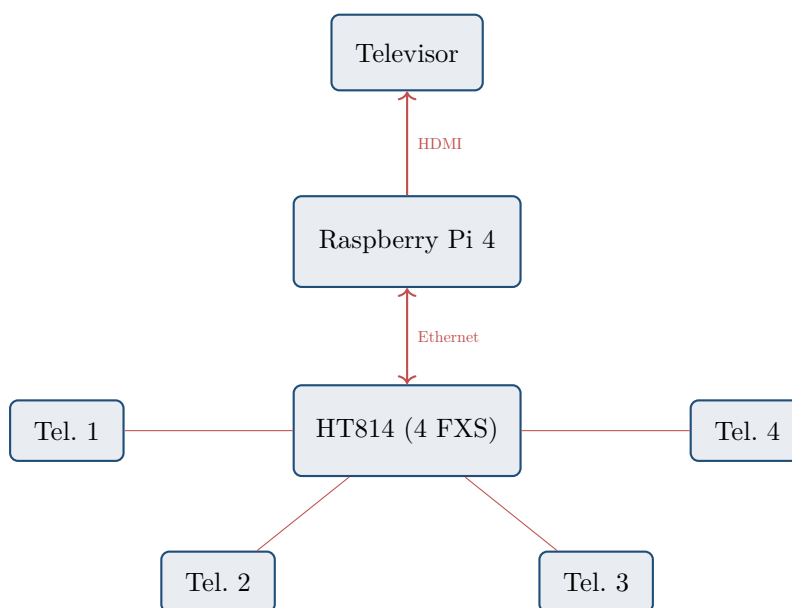
```

Crea un dispositivo virtual `hw:Loopback,0`. DOSBox se configura para sacar audio a ese dispositivo. Asterisk abre el otro extremo del loopback como un canal de entrada y lo *bridgea* a la sala.

9.5.2 Camino B: PipeWire

Más flexible y moderno pero con mayor consumo de recursos. En la Pi 4 (2 GB) podría ser un cuello de botella; en Pi 5 funciona sin problema.

9.6 Topología física



9.7 Funcionalidades opcionales

9.7.1 Identificación del jugador

Al inicio de cada llamada, antes de entrar a la sala, se puede pedir al jugador que marque un dígito para identificarse:

```
"Bienvenido a Hugo. Marca 1 si eres nuevo, o tu numero de jugador  
si ya tienes uno."
```

Esto permite llevar un ranking persistente entre sesiones.

9.7.2 Marcador / ranking

Una base SQLite local en la Pi puede almacenar puntajes por jugador. Cada vez que Hugo registra una vida perdida o un nivel completado (información que podría capturarse vía OCR sobre la salida HDMI o leyendo memoria del proceso DOSBox), el marcador se actualiza.

9.7.3 Display de marcador en pantalla

Un overlay puede dibujarse sobre la salida HDMI mediante:

- OBS Studio en modo composición.
- Un segundo proceso que dibuje con SDL2 sobre una capa transparente.
- Una pantalla LCD secundaria pequeña dedicada al marcador.

9.7.4 Streaming a Twitch o YouTube

Con una capturadora HDMI USB conectada a la Mac (no a la Pi, para no sobrecargarla), se puede transmitir las partidas. Esto permite a otros amigos ver el juego sin necesidad de estar físicamente en el lugar.

9.8 Estimación de esfuerzo

Componente	Esfuerzo aprox.
ConfBridge y nuevas extensiones SIP	4 horas
Lógica de turnos (Rust)	16 horas
Audio loopback ALSA	4 horas
Identificación de jugadores	4 horas
Persistencia SQLite	4 horas
Overlay de marcador	12 horas
Pruebas y calibración	8 horas
Total estimado	~52 horas

Tabla 9.1: Estimación de tiempo para implementar el modo multijugador.

9.9 Cierre

El sistema base diseñado en los capítulos 4–8 está pensado para crecer hacia esta versión multijugador sin necesidad de reescribir componentes. Cuando llegue el momento de implementarla, se debe abrir una nueva fase de diseño detallada que tome estas bases como referencia.

Referencia técnica

10.1 Mapeo DTMF → teclado

DTMF	Frecuencias (Hz)	Tecla	Código evdev	Acción en Hugo
1	697 + 1209	Q	KEY_Q	libre / acción 1
2	697 + 1336	↑	KEY_UP	saltar o subir
3	697 + 1477	E	KEY_E	libre / acción 2
4	770 + 1209	←	KEY_LEFT	izquierda
5	770 + 1336	Espacio	KEY_SPACE	acción principal
6	770 + 1477	→	KEY_RIGHT	derecha
7	852 + 1209	Z	KEY_Z	libre / acción 3
8	852 + 1336	↓	KEY_DOWN	bajar / agacharse
9	852 + 1477	X	KEY_X	libre / acción 4
*	941 + 1209	Shift Izq.	KEY_LEFTSHIFT	modificador
0	941 + 1336	Esc	KEY_ESC	pausa / menú
#	941 + 1477	Enter	KEY_ENTER	confirmar

Tabla 10.1: Mapeo DTMF → teclado por defecto.

10.2 Archivos clave del sistema

Archivo	Función
/etc/asterisk/pjsip.conf	Define extensiones SIP 101 y 102 (chan_pjsip)
/etc/asterisk/extensions.conf	Dialplan: lógica del 9000
/etc/asterisk/manager.conf	Habilita AMI con usuario hugo-bridge
/etc/systemd/system/hugo-bridge.service	Unit del servicio bridge
/etc/udev/rules.d/99-hugo-uinput.rules	Permisos sobre /dev/uinput
/opt/hugo-phone/bridge/hugo-bridge	Binario compilado del bridge
/opt/hugo-phone/bridge/config.toml	Configuración del bridge (hot-reload)
/opt/hugo-phone/games/hugo/	Archivos del juego
/opt/hugo-phone/games/dosbox-hugo.conf	Configuración del emulador (invocada con dosbox -conf)

10.3 Servicios y comandos útiles

10.3.1 Gestión de Asterisk

```
1 sudo systemctl status asterisk
2 sudo systemctl restart asterisk
3 sudo asterisk -rvvv           # consola interactiva
4 sudo asterisk -rx "reload"    # recargar configs sin reiniciar
```

Comandos dentro del CLI de Asterisk:

```

pjsip show endpoints      ; estado de extensiones
pjsip show contacts      ; clientes registrados (AOR contacts)
pjsip set logger on/off   ; trazas SIP detalladas
dialplan show hugo-game   ; ver dialplan cargado
manager show settings     ; estado de AMI
manager show users        ; usuarios AMI
manager show connected    ; sesiones AMI activas
core show channels        ; llamadas activas

```

10.3.2 Gestión del bridge

```

1 sudo systemctl status hugo-bridge
2 sudo systemctl restart hugo-bridge
3 sudo journalctl -u hugo-bridge -f      # logs en vivo
4 sudo journalctl -u hugo-bridge -n 100  # ultimas 100 lineas

```

10.3.3 Recompilar el bridge

```

1 cd /opt/hugo-phone/bridge
2 sudo -u beto ~/.cargo/bin/cargo build --release
3 sudo cp target/release/hugo-bridge ./hugo-bridge
4 sudo systemctl restart hugo-bridge

```

10.3.4 Inspección del teclado virtual

```

1 ls /dev/input/by-id/          # buscar hugo-phone-...
2 sudo evtest                  # seleccionar el dispositivo
3                               # ver eventos al marcar DTMF

```

10.3.5 Verificación del HT801

```

1 ping 192.168.1.11            # alcanzabilidad IP
2 nmap -p 5060 192.168.1.11    # puerto SIP abierto

```

10.4 Estructura del archivo config.toml

```

1 [ami]
2 host = "127.0.0.1"
3 port = 5038
4 username = "hugo-bridge"
5 secret = "PASSWORD-REAL"
6
7 [bridge]
8 press_duration_ms = 80
9 event_name = "HugoKey"
10 key_field = "Key"
11
12 [keymap]

```



```
13 "2" = "Up"
14 "4" = "Left"
15 "6" = "Right"
16 "8" = "Down"
17 "5" = "Space"
18 "0" = "Esc"
19 "1" = "Q"
20 "3" = "E"
21 "7" = "Z"
22 "9" = "X"
23 "*" = "LeftShift"
24 "#" = "Enter"
```

10.5 Nombres de teclas soportados

El parser de `keyboard.rs` reconoce:

- **Direccionales:** Up, Down, Left, Right
- **Especiales:** Space, Enter, Esc (o Escape), Tab, Backspace
- **Modificadores:** LeftShift, RightShift, LeftCtrl, RightCtrl, LeftAlt, RightAlt
- **Letras:** A a Z
- **Función:** F1 a F12

Si se asigna un nombre no reconocido, el bridge registra un warning en los logs pero sigue funcionando.

10.6 Anatomía del protocolo AMI

10.6.1 Banner inicial

```
Asterisk Call Manager/9.0.0
```

10.6.2 Login

```
Action: Login
Username: hugo-bridge
Secret: PASSWORD-REAL
Events: user,call
(linea en blanco)
```

10.6.3 Respuesta exitosa de login

```
Response: Success
Message: Authentication accepted
(linea en blanco)
```

10.6.4 UserEvent producido por el dialplan

```
Event: UserEvent
Privilege: user,all
UserEvent: HugoKey
Channel: PJSIP/101-00000003
Caller: 101
Key: 2
Uniqueid: 1715641234.3
(linea en blanco)
```

10.7 Glosario

AMI *Asterisk Manager Interface*. Protocolo TCP de texto plano para controlar Asterisk desde clientes externos.

ATA *Analog Telephone Adapter*. Dispositivo que convierte una línea analógica a SIP.

DOSBox Emulador del sistema operativo MS-DOS y del hardware del PC compatible (CPU, VGA, Sound Blaster, etc.).

DTMF *Dual-Tone Multi-Frequency*. Sistema de señalización por pares de tonos, usado por los teclados telefónicos.

evdev Subsistema del kernel Linux que expone dispositivos de entrada a *user space*.

FXO *Foreign eXchange Office*. Puerto que se conecta a una central como si fuera un teléfono.

FXS *Foreign eXchange Subscriber*. Puerto que entrega línea como si fuera la compañía telefónica.

KMSDRM *Kernel Mode Setting / Direct Rendering Manager*. Capa del kernel Linux que permite dibujar directamente a la pantalla sin servidor X.

PBX *Private Branch Exchange*. Centralita telefónica privada.

RFC 2833 Estándar para transportar tonos DTMF dentro de paquetes RTP, fuera del audio principal.

RTP *Real-time Transport Protocol*. Protocolo para transmitir audio y video en tiempo real, sobre UDP.

SIP *Session Initiation Protocol*. Protocolo de señalización para iniciar, mantener y terminar sesiones de voz/video.

uinput Subsistema del kernel Linux que permite crear dispositivos de entrada virtuales desde *user space*.

10.8 Referencias externas

- Documentación oficial de Asterisk: wiki.asterisk.org

- DOSBox: www.dosbox.com
- Crate uinput de Rust: docs.rs/uinput
- Tokio: tokio.rs
- Grandstream HT801 manual: grandstream.com/products
- Raspberry Pi documentation: raspberrypi.com/documentation
- Hugo en MyAbandonware: myabandonware.com/game/hugo-eya

10.9 Créditos

Hugo, el personaje y la franquicia, son propiedad de **ITE Media ApS** (originalmente *Interactive Television Entertainment*), Dinamarca. Este proyecto es un homenaje técnico sin fines de lucro y sin afiliación oficial con la compañía.

El sistema completo, su código y su documentación fueron desarrollados como proyecto personal por **Beto** en colaboración con Claude (Anthropic) durante mayo de 2026.

“Hugo, ¡cuidado con la roca!”
— *Generaciones de niños chilenos en los noventa*